

Optimal Trajectory Tracking and Obstacle Avoidance for Differential-Drive Mobile Robots

Serkan Basaran
Helin Geleri
Muhammet Emre Eren

M.Sc. Programme in Automation and Control Engineering
Numerical Optimization for Control, Prof. Lorenzo Mario Fagiano

February 16, 2026

Abstract

This report addresses the problem of optimal trajectory tracking with obstacle avoidance for a differential-drive mobile robot. The control objective is first formulated as an unconstrained and then as a constrained optimization problem, where system dynamics, input limitations, and collision avoidance constraints are explicitly taken into account. Numerical optimization techniques are used to compute feasible and optimal control inputs. The simulation results illustrate the effectiveness of the proposed approach.

Keywords: Numerical optimization, trajectory tracking, obstacle avoidance, mobile robots, optimal control.

Contents

1	Introduction	5
2	System Modeling	5
2.1	Differential-Drive Robot Model	6
2.1.1	Configuration and Nonholonomic Constraint	6
2.1.2	Unicycle Kinematic Model	7
2.1.3	Differential-Drive Wheel Mapping	7
2.1.4	Accurate Nonlinear Dynamic Model	7
2.1.5	Simplified wheel dynamics	9
2.1.6	Actuator saturation	9
2.2	Discretization of the Continuous-Time Model and Sampling Time Selection	9
2.3	Practical Implementation Suggestions	13
3	Part I: Unconstrained Formulation with Barrier Functions	14
3.1	Barrier Function Candidate and Selection	14
3.2	Performance Index	14
3.3	Quasi-Newton BFGS Optimization	14
3.4	Optimal Step Size via Strong Wolfe Conditions	15
3.5	Numerical Gradient Computation	15
3.6	Implementation Workflow	15
3.7	Simulation Results	16
3.7.1	Trajectory and Navigation Analysis	17
3.7.2	Robot and Wheel Velocities Analysis	17
3.7.3	Actuator Dynamics and Constraints	18
3.7.4	Cost Function Convergence	18
4	Part II: Constrained Optimal Control Problem	19
4.1	Constrained MPC Formulation	20
4.1.1	Objective Function and Constraints	21
4.1.2	Basic Configurations on Circular Static Constraints	21
4.1.3	Linearized Static Obstacle Constraints	22
4.1.4	Ellipsoid Approximation for Rectangular Obstacles	23
4.1.5	Minimum–Area Ellipse Formulation	23
4.1.6	Linearization of the Ellipsoidal Constraint	24
4.1.7	Dynamic Obstacle Avoidance via Control Barrier Functions (CBF)	25

4.1.8	Preliminaries: Lie Derivatives and Class- $\mathcal{K}$ Functions	26
4.1.9	Continuous-Time CBF Formulation	26
4.1.10	Relative Degree Analysis	27
4.1.11	High-Order (Second-Order) Continuous-Time CBF	27
4.1.12	Discrete-Time CBF Formulation	28
4.1.13	Discrete-Time High-Order CBF (Relative Degree m)	28
4.1.14	Relative Degree Two Case	29
4.1.15	Integration into the MPC Framework	29
4.2	Simulation Results	29
4.2.1	Trajectory and Constraint Satisfaction	30
4.2.2	Robot and Wheel Velocities Analysis	30
4.2.3	Actuator Torques and Feasibility of Hard Constraint	31
4.2.4	Cost Function Convergence	32
4.2.5	Comparison with Unconstrained Formulation	33
4.2.6	Artificial LiDAR Integration	33
5	Numerical Optimization Strategy	33
6	Conclusions	34

Notation

Symbol	Description	Unit
t	Time variable	[s]
$z(t)$	State vector $[x, y, \theta, \omega_r, \omega_l]^T$	—
$u(t)$	Control input vector $[\tau_r, \tau_l]^T$	—
(x, y)	Coordinates of the robot center of gravity (COG)	[m]
θ	Heading angle relative to the X-axis	[rad]
ω_r, ω_l	Angular velocities of the right and left wheels	[rad/s]
τ_r, τ_l	Torque applied to the right and left wheels	[N m]
v, w	Linear and angular velocity of the robot body	[m/s, rad/s]
r	Wheel radius	[m]
L	Half of the wheelbase distance	[m]
d	Distance between wheel axis and COG (Offset)	[m]
M	Total mass of the robot	[kg]
J_b	Body inertia moment about the COG	[kg m ²]
J_w	Moment of inertia of each wheel	[kg m ²]
B_w	Wheel viscous friction coefficient	[N m s/rad]
τ_{max}	Maximum actuator torque saturation limit	[N m]
Inertial Coefficients (Auxiliary Variables)		
A_1	Effective translational mass	[kg]
A_2	Effective rotational inertia	[kg m ²]
A_3	Inertial coupling term ($M \cdot d$)	[kg m]

1 Introduction

Autonomous navigation of mobile robots in cluttered environments remains a cornerstone of modern robotics, requiring the simultaneous fulfillment of precise trajectory tracking and safety-critical obstacle avoidance. This project investigates the application of numerical optimization techniques to a differential-drive mobile robot, characterized by non-holonomic constraints and nonlinear dynamics.

This research is structured in two distinct phases to evaluate different optimization philosophies. In the first phase, we develop an unconstrained optimal control framework where obstacle avoidance is integrated into the performance index through the use of exponential barrier functions. This approach simplifies the optimization landscape, allowing for the use of efficient quasi-Newton solvers with BFGS updates to compute optimal trajectories that naturally deviate from obstacles as the cost increases in their proximity.

In the second phase, the framework is extended to a more robust, explicitly constrained formulation of model predictive control (MPC). The limitations of barrier-based methods are addressed by enforcing safety through hard constraints. This phase introduces significant technical improvements, including the linearization of static circular obstacle constraints and the use of elliptic approximations for rectangular obstacles to maintain differentiability. Furthermore, we propose a Control Barrier Function (CBF) approach to manage dynamic obstacles, ensuring forward invariance of the safe set even in time-varying environments.

The objective is to minimize a cost function that includes tracking accuracy, control energy (torque), and smoothness of motion, while ensuring that the robot navigates safely through a series of goals. By comparing simplified and accurate nonlinear dynamic models across various sampling times and integration methods (e.g., Forward Euler vs. Runge-Kutta 4), this report provides a comprehensive analysis of the trade-offs between modeling fidelity and computational efficiency.

2 System Modeling

This section presents the modeling framework adopted for the differential-drive mobile robot (DDMR). The key modeling choices are outlined, including the selection of system states and control inputs, the choice between simplified and accurate dynamic models, the geometric representation and physical parameters of the robot, and the discretization strategy used for numerical implementation.

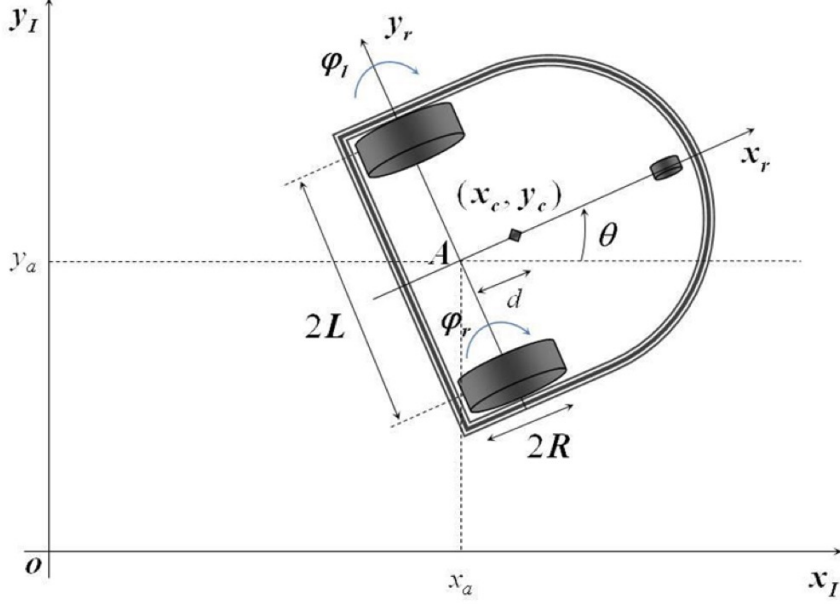


Figure 1: Kinematic and dynamic representation of the differential-drive mobile robot, illustrating the coordinate frames, mechanical parameters (L, r) .

2.1 Differential-Drive Robot Model

The mathematical representation of the differential-drive mobile robot (DDMR) is fundamental to the design of the optimal control law. We describe the system through two interconnected layers: the kinematic layer, which includes the geometry of motion, and the dynamic layer, which relates applied torques to physical accelerations. This dual-layer approach allows for a comprehensive understanding of how low-level actuator signals translate into high-level trajectory changes.

2.1.1 Configuration and Nonholonomic Constraint

The planar pose of the mobile robot is described by the configuration vector as

$$q = [x \ y \ \theta]^T, \quad (1)$$

where (x, y) denotes the position of the midpoint of the axle of the wheel expressed in the inertial frame (world), and θ is the angle of travel. The general geometric and dynamic configuration adopted in this section is illustrated in Figure 1. For the specific robot considered in this project, the center of mass is later assumed to coincide with the midpoint of the axle, i.e. $d = 0$, and the platform is modeled as a circular symmetric body.

Since the robot is a wheeled vehicle under the assumption of pure rolling, lateral slip is neglected. This yields the nonholonomic rolling constraint, stating that the velocity component orthogonal to the robot heading must be zero:

$$-\sin \theta \dot{x} + \cos \theta \dot{y} = 0. \quad (2)$$

The constraint in (2) restricts the admissible instantaneous velocities of the robot, but cannot be integrated into an algebraic constraint involving only (x, y, θ) , hence it is nonholonomic.

2.1.2 Unicycle Kinematic Model

Let v denote the longitudinal velocity of the robot expressed in its body frame, and let ω denote the yaw rate. Under the nonholonomic constraint (2), the kinematic equations of motion take the standard unicycle form:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega. \quad (3)$$

2.1.3 Differential-Drive Wheel Mapping

The robot is equipped with two independently actuated wheels. Let r denote wheel radius and L the *half-track* (so that the wheel-to-wheel distance is $2L$). Letting ω_R and ω_L denote the angular velocities of the right and left wheels, respectively, we denote the full state vector in this work as

$$\mathbf{x} = [x \ y \ \theta \ \omega_r \ \omega_l]^T. \quad (4)$$

Therefore, we use the wheel mapping to express the inertial-frame velocities. The corresponding linear velocities of the wheels are

$$v_R = r\omega_R, \quad v_L = r\omega_L. \quad (5)$$

For a differential-drive geometry, the body-frame forward velocity and yaw rate are given by

$$v = \frac{v_R + v_L}{2} = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{v_R - v_L}{2L} = \frac{r}{2L}(\omega_R - \omega_L). \quad (6)$$

Substituting (6) into (3) yields the kinematic model of the differential-drive robot in the inertial frame, relating the right and left wheel angular velocities to the mobile robot velocities in the inertial frame.

$$\dot{x} = \frac{r}{2}(\omega_R + \omega_L) \cos \theta, \quad (7)$$

$$\dot{y} = \frac{r}{2}(\omega_R + \omega_L) \sin \theta, \quad (8)$$

$$\dot{\theta} = \frac{r}{2L}(\omega_R - \omega_L). \quad (9)$$

2.1.4 Accurate Nonlinear Dynamic Model

This section summarizes the differential-drive robot dynamics adopted in this work, following the unified framework in [1]. Two equivalent derivations are presented in [1]: a constrained Lagrange formulation (with elimination of Lagrange multipliers via a null-space velocity transformation) and a Newton-Euler formulation (via force/moment balance in the body frame under non-holonomic constraints). Both approaches lead to the same reduced dynamics, which we implement in a state-space form with wheel angular velocities as states.

The accurate model accounts for platform mass M , yaw inertia about the COM J_b , wheel/motor rotational inertia about the wheel axis J_w , and a possible longitudinal COM offset d from the axle midpoint A (COM is at distance d along the symmetry axis). The key nonlinear coupling terms originate from $d \neq 0$ and appear as $-Md\omega^2$ and $+Md v\omega$ in the translational and yaw dynamics, respectively [1].

Lagrange approach (constraint elimination). In [1], the generalized coordinates include the wheel angles and the planar pose; the non-holonomic constraints are enforced with Lagrange multipliers and then eliminated by expressing $\dot{\mathbf{q}} = S(\mathbf{q})\eta$ with $\eta = [\dot{\phi}_R \ \dot{\phi}_L]^T = [\omega_r \ \omega_l]^T$ spanning the null-space of the constraint matrix. This yields reduced dynamics in the wheel-rate coordinates of the form

$$M_\eta(\cdot) \dot{\omega} + h_\eta(\cdot) = \tau, \quad (10)$$

which can be equivalently expressed in the body velocities (v, ω) [1]. In this work we implement the explicit (v, ω) form and map it back to $\dot{\omega}_r, \dot{\omega}_l$ (below), which is algebraically equivalent to (10).

Newton–Euler approach (force/moment balance). In [1], applying Newton’s second law in the robot frame and enforcing the non-holonomic constraints leads to coupled nonlinear equations in the body linear and angular velocities. Incorporating the wheel inertia consistently with the reduced Lagrangian model results in the compact form

$$A_1 \dot{v} - A_3 \omega^2 = \frac{1}{r} (\tau_r + \tau_l), \quad (11)$$

$$A_2 \dot{\omega} + A_3 v \omega = \frac{L}{r} (\tau_r - \tau_l), \quad (12)$$

where the effective inertial coefficients are

$$A_1 = M + \frac{2J_w}{r^2}, \quad A_2 = J_b + Md^2 + \frac{2J_w L^2}{r^2}, \quad A_3 = Md. \quad (13)$$

Solving (11)–(12) yields

$$\dot{v} = \frac{\frac{1}{r}(\tau_r + \tau_l) + A_3 \omega^2}{A_1}, \quad (14)$$

$$\dot{\omega} = \frac{\frac{L}{r}(\tau_r - \tau_l) - A_3 v \omega}{A_2}. \quad (15)$$

Wheel acceleration dynamics $\dot{\omega}_r, \dot{\omega}_l$ (state-space form). Since the MPC state includes ω_r, ω_l , we obtain explicit wheel-rate derivatives by differentiating the inverse kinematics

$$\omega_r = \frac{v + L\omega}{r}, \quad \omega_l = \frac{v - L\omega}{r}, \quad (16)$$

which gives

$$\dot{\omega}_r = \frac{\dot{v} + L\dot{\omega}}{r}, \quad \dot{\omega}_l = \frac{\dot{v} - L\dot{\omega}}{r}. \quad (17)$$

Substituting (14)–(15) into (17), with v and ω computed from (??), yields the closed-form dynamics

$$\dot{\omega}_r = \frac{1}{r} \left(\frac{\frac{1}{r}(\tau_r + \tau_l) + A_3 \omega^2}{A_1} + L \frac{\frac{L}{r}(\tau_r - \tau_l) - A_3 v \omega}{A_2} \right), \quad (18)$$

$$\dot{\omega}_l = \frac{1}{r} \left(\frac{\frac{1}{r}(\tau_r + \tau_l) + A_3 \omega^2}{A_1} - L \frac{\frac{L}{r}(\tau_r - \tau_l) - A_3 v \omega}{A_2} \right). \quad (19)$$

Additional viscous wheel/actuator losses. The derivations in [1] focus on the mechanical coupling and non-holonomic constraints. We additionally model small viscous losses at each wheel/actuator by an effective torque

$$\tau_r^{\text{eff}} = \tau_r - B_w \omega_r, \quad \tau_l^{\text{eff}} = \tau_l - B_w \omega_l, \quad (20)$$

and used in (14)–(19) by replacing τ_r, τ_l with $\tau_r^{\text{eff}}, \tau_l^{\text{eff}}$.

2.1.5 Simplified wheel dynamics

As a low-fidelity baseline, we also consider, as a second model, decoupled wheel dynamics with viscous damping,

$$\dot{\omega}_r = \frac{\tau_r - B_w \omega_r}{J_w}, \quad \dot{\omega}_l = \frac{\tau_l - B_w \omega_l}{J_w}, \quad (21)$$

which neglects platform inertia and the coupling induced by M , J_b , and d .

2.1.6 Actuator saturation

The commanded torques are saturated according to

$$\tau_i = \text{sat}(\tau_i), \quad |\tau_i| \leq \tau_{\max}, \quad i \in \{r, l\}, \quad (22)$$

with $\tau_{\max} = 2.85$ Nm in our setup.

Finally, the complete state-space model used in this work is

$$\dot{x} = v \cos \theta, \quad (23)$$

$$\dot{y} = v \sin \theta, \quad (24)$$

$$\dot{\theta} = \omega, \quad (25)$$

$$\dot{\omega}_r = f_r(\omega_r, \omega_l, \tau_r, \tau_l), \quad (26)$$

$$\dot{\omega}_l = f_l(\omega_r, \omega_l, \tau_r, \tau_l), \quad (27)$$

where $f_r(\cdot)$ and $f_l(\cdot)$ are obtained from the reduced dynamics in (18) - (19) for the accurate model, and from (21) for the simplified model.

The input motor torques are subject to saturation limits

$$\tau_i = \text{sat}(\tau_i), \quad |\tau_i| \leq \tau_{\max}. \quad (28)$$

2.2 Discretization of the Continuous-Time Model and Sampling Time Selection

The differential-drive robot dynamics derived in the previous section are formulated in continuous time as a nonlinear ordinary differential equation (ODE) of the form

$$\dot{x}(t) = f(x(t), u(t)), \quad (29)$$

where $x(t)$ denotes the state vector and $u(t)$ the control input. However, since the considered simulation and optimal control framework operates in discrete time with a finite sampling period T_s , it is necessary to obtain a discrete-time representation of the model.

Given a sampling time T_s , the discrete-time state is defined as $x_k = x(kT_s)$. The exact evolution of the system over one sampling interval can be written in integral form as

$$x_{k+1} = x_k + \int_{kT_s}^{(k+1)T_s} f(x(t), u(t)) dt, \quad (30)$$

which represents the general solution of the continuous-time ODE. In practice, the integral in (30) cannot be evaluated analytically for nonlinear systems, since the state trajectory $x(t)$

within the interval is unknown. Therefore, a numerical integration method is required in order to approximate the state propagation map and obtain a discrete-time model of the form

$$x_{k+1} \approx F(x_k, u_k). \quad (31)$$

In this work, Forward Euler and the classical 4th-order Runge–Kutta (RK4) scheme are considered as the numerical integration schemes to propagate the continuous-time dynamics and assess the impact of the sampling time on simulation accuracy. Under the assumption that the input is constant over the sampling interval (zero-order hold), the RK4 update is given by

$$x_{k+1} = x_k + \frac{T_s}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (32)$$

where

$$k_1 = f(x_k, u_k), \quad (33)$$

$$k_2 = f\left(x_k + \frac{T_s}{2} k_1, u_k\right), \quad (34)$$

$$k_3 = f\left(x_k + \frac{T_s}{2} k_2, u_k\right), \quad (35)$$

$$k_4 = f(x_k + T_s k_3, u_k). \quad (36)$$

To investigate the influence of discretization in a realistic navigation scenario, a reference control sequence was first generated by solving the nonlinear MPC problem using the accurate continuous-time dynamic model, discretized with RK4 and a small sampling time $T_s^{\text{ref}} = 0.001$ s. This configuration yields a dynamically feasible obstacle-avoidance trajectory on a nontrivial map and is therefore adopted as a benchmark solution. The exact configuration of goals and obstacles, with the path of the computed optimal solution can be seen in Figure 2.

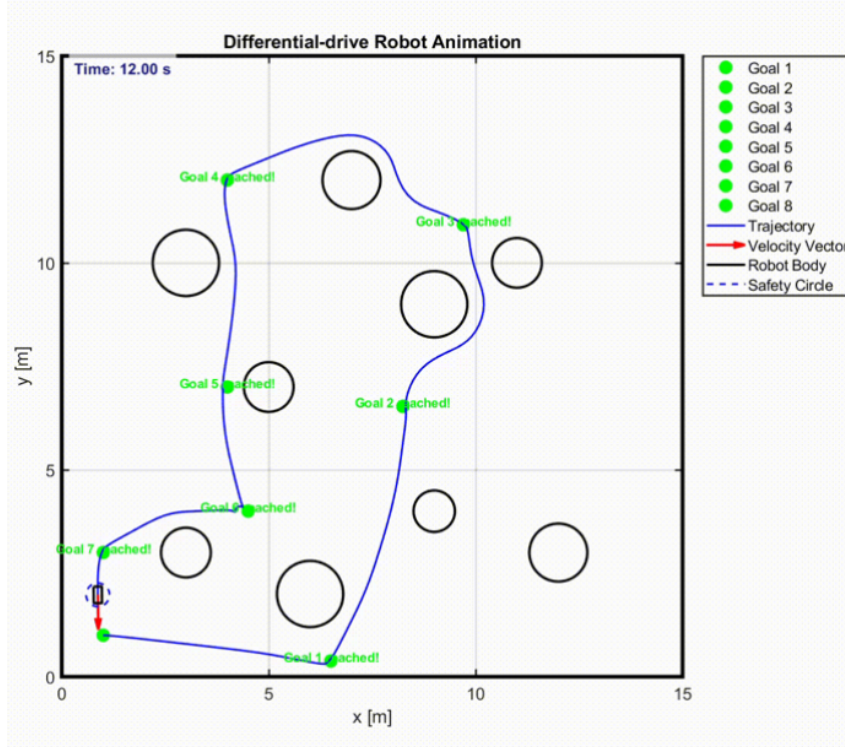


Figure 2: Example navigation scenario chosen for discretization comparisons

The recorded input sequence is subsequently replayed in open loop using different discretization settings. In these simulations, the discretization step is chosen equal to the assumed control update period ($h = T_s$). As a result, increasing T_s simultaneously affects both the numerical propagation of the dynamics and the temporal resolution at which the precomputed control signal is effectively applied under a zero-order hold assumption.

Consequently, the deviation from the reference trajectory captures the combined effect of:

- numerical discretization of the continuous-time dynamics, and
- reduced temporal resolution of the applied control sequence.

This was therefore interpreted as a combined study of numerical integration error and an assessment of the sensitivity of the nominal optimal motion to a coarser discrete-time implementation.

Since the replay experiment is performed in open loop, deviations accumulate over time without feedback correction. In a true receding-horizon closed-loop NMPC implementation, the control input would be recomputed at every sampling instant, and feedback would mitigate part of the observed drift. The present analysis thus provides a conservative sensitivity assessment and serves to identify candidate sampling times that preserve the qualitative structure and feasibility of the trajectory. Final validation of the selected sampling time was therefore ultimately performed in closed-loop NMPC simulations.

In order to select an appropriate sampling time T_s for simulation and control design, we evaluated the sensitivity of the generated trajectories with respect to both the numerical integration scheme and the discretization step size. The benchmark reference trajectory described above is assumed to be sufficiently close to the continuous-time behavior of the accurate model.

Subsequently, the same input sequence was simulated using a grid of discretization settings consisting of:

- Forward Euler and RK4 integration methods,
- Sampling times $T_s \in \{0.001, 0.01, 0.05, 0.06, 0.075, 0.1\}$ s,
- Two different continuous-time dynamic models: an accurate model and a simplified model.

For each simulation, the position root-mean-square error (RMSE) with respect to the golden reference trajectory was computed as

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N \left\| \begin{bmatrix} x_k \\ y_k \end{bmatrix} - \begin{bmatrix} x_k^{\text{ref}} \\ y_k^{\text{ref}} \end{bmatrix} \right\|_2^2}, \quad (37)$$

where (x_k, y_k) denotes the simulated planar position and $(x_k^{\text{ref}}, y_k^{\text{ref}})$ denotes the corresponding golden reference position.

In addition, an *inter-model RMSE* was computed between the accurate and simplified model trajectories at equal sampling time, defined as

$$\text{RMSE}_{\text{inter}} = \sqrt{\frac{1}{N} \sum_{k=1}^N \left\| \begin{bmatrix} x_k^{\text{acc}} \\ y_k^{\text{acc}} \end{bmatrix} - \begin{bmatrix} x_k^{\text{simp}} \\ y_k^{\text{simp}} \end{bmatrix} \right\|_2^2}. \quad (38)$$

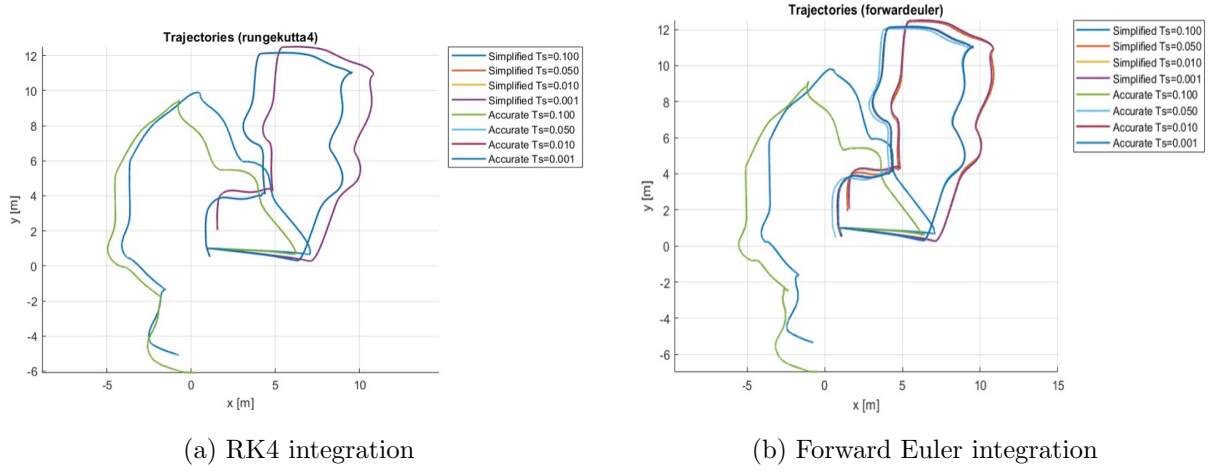


Figure 3: Open-loop simulation paths for different sampling times and dynamic models. Comparison between RK4 (left) and Forward Euler (right) integration schemes.

A slight difference in acceleration causes an accumulation of the difference in speed as it is integrated, which in turn causes an accumulation of the difference in pose. This causes the whole path to drift. Figure 3 shows the resulting paths for the RK4 and Euler integration schemes for the simplified and accurate models over different sampling times. The obtained results show that decreasing the sampling time significantly reduces the trajectory error for both Euler and RK4 schemes. This behavior is expected, since the discretization error vanishes as $T_s \rightarrow 0$, and both numerical integration methods converge to the corresponding continuous-time solution. In particular, RK4 achieves substantially lower errors at moderate sampling times due to its higher order of accuracy.

Table 1: Open-loop RMSE comparison for different sampling times (RK4 integration).

T_s [s]	Simplified		Accurate		Inter-model	
	RMSE [m]	(%)	RMSE [m]	(%)	RMSE [m]	(%)
0.100	6.6409	69.33	7.2608	83.22	1.1534	13.22
0.075	4.3982	45.91	4.9114	56.29	1.2458	14.28
0.060	2.3982	25.04	2.7152	31.12	1.4450	16.56
0.050	0.0744	0.78	0.0702	0.80	1.6279	18.66
0.010	0.0139	0.15	0.0132	0.15	1.6286	18.67
0.001	0.0000	0.00	0.0000	0.00	1.6291	18.67

Table 2: Open-loop RMSE comparison for different sampling times (Forward Euler integration).

T_s [s]	Simplified		Accurate		Inter-model	
	RMSE [m]	(%)	RMSE [m]	(%)	RMSE [m]	(%)
0.100	6.7217	70.17	7.3603	84.37	1.4316	16.41
0.075	4.4549	46.50	4.9314	56.53	1.4421	16.53
0.060	2.4675	25.76	2.7310	31.30	1.6016	18.36
0.050	0.1544	1.61	0.1451	1.66	1.7550	20.12
0.010	0.0277	0.29	0.0244	0.28	1.6514	18.93
0.001	0.0000	0.00	0.0000	0.00	1.6313	18.70

The tables 1 and 2 show the RMSE position errors for each model, sampling time and integration scheme combination; and the inter-model RMSE error, that is, the error between the two models for the given sampling time and integration scheme combination. A key observation is that the inter-model RMSE $\text{RMSE}_{\text{inter}}$ converges to a non-zero constant value as T_s decreases. This indicates that for sufficiently small T_s , the dominant discrepancy is no longer due to numerical integration, but rather due to an intrinsic modeling mismatch between the simplified and accurate continuous-time dynamics. In other words, while discretization errors vanish as $T_s \rightarrow 0$, the difference between the two models reduces to their representations of different physical assumptions and parameterizations.

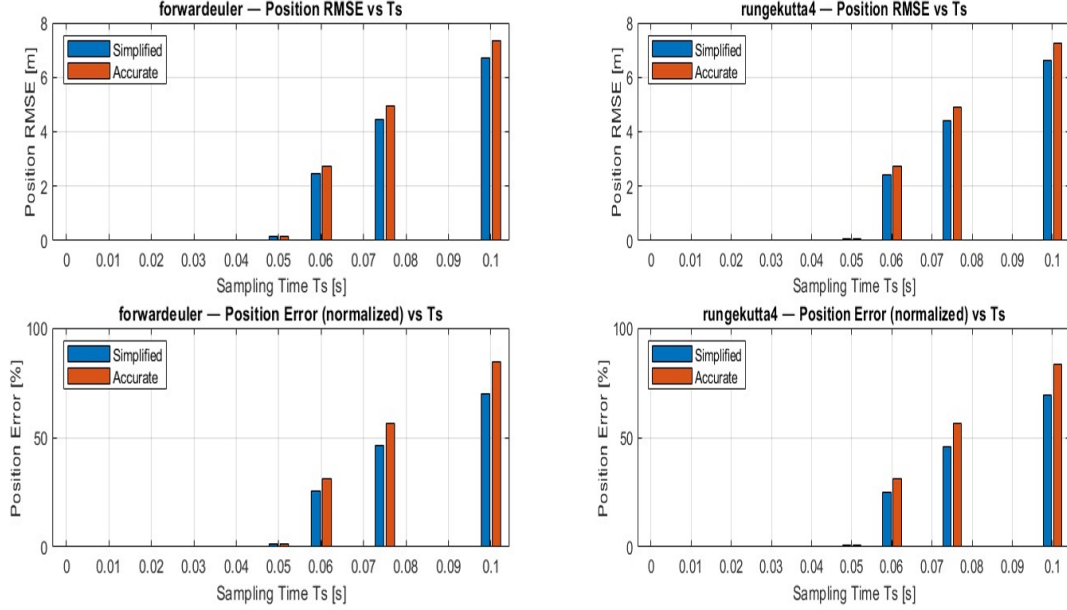


Figure 4: RMSE position error vs different sampling times for Euler and RK4 schemes

Based on the above results, and on the plot of RMSE vs T_s in Figure 4, a knee point in $T_s = 0.05$ s is observed. Therefore, a sampling time of $T_s = 0.05$ s was selected as a compromise between numerical accuracy and computational efficiency. For this value, both Euler and RK4 produce sufficiently accurate trajectories with respect to the golden reference, while further decreasing T_s provides diminishing returns compared to the residual modeling mismatch between the simplified and accurate dynamics.

2.3 Practical Implementation Suggestions

Transitioning this theoretical model to a real-world application involves addressing challenges in perception and hardware communication. We suggest the following enhancements:

- **Perception via LiDAR:** In a physical environment, the robot cannot "see" the obstacles directly. Integrating an artificial LiDAR range system would allow the robot to perform distance-to-obstacle measurements in real-time, effectively feeding the constraints with live data and improving local path planning.
- **Communication via ROS2:** For modern robotics, an ROS2 environment (Robot Operating System 2) is the ideal intermediate layer. It would facilitate asynchronous communication between the high-level optimization node (solving the MPC) and the low-level

motor controllers, ensuring high reliability and modularity. It would also give the opportunity to consider sensor data realistic.

3 Part I: Unconstrained Formulation with Barrier Functions

In the first phase of this project, the obstacle avoidance requirement is handled by transforming the constrained control problem into an unconstrained nonlinear optimization problem. This is achieved by augmenting the performance index with smooth, continuously differentiable barrier functions that penalize proximity to obstacles.

3.1 Barrier Function Candidate and Selection

To enforce the safety region without explicit constraints, an *exponential barrier function* (39) is utilized. Unlike the logarithmic barrier, the exponential form provides a faster "trigger" rate as the robot approaches an obstacle boundary, which is critical for high-speed maneuvering and real-time collision avoidance. The obstacle cost for a given state z_k is defined as:

$$J_{obs}(z_k) = \sum_{i=1}^{N_{obs}} \alpha \cdot \exp(-\beta \cdot d_i(z_k)) \quad (39)$$

where $d_i(z_k) = \|z_{1:2} - O_i\|_2 - R_i - r_{robot}$ is the safety distance to the i -th obstacle, and $\alpha = 1000, \beta = 50$ are positive constant tuning parameters for penalty magnitude and rate scaling respectively. Since the Euclidean distance norm is convex and the exponential of a negative convex function is non-convex, the whole problem, as discussed in Section 3.2 (40), can be stated as a non-convex unconstrained optimal control problem.

3.2 Performance Index

The total cost function J (40) over a prediction horizon N_p is formulated to penalize position tracking error, control effort, control smoothness, wheel angular velocities, and obstacle avoidance cost:

$$\min_U \sum_{k=0}^{N_p-1} W_{pos} \|p_k - p_{goal}\|_2^2 + W_u \|u_k\|_2^2 + W_{\Delta u} \|\Delta u_k\|_2^2 + W_{\omega_r} \|\omega_{r,k}\|_2^2 + W_{\omega_l} \|\omega_{l,k}\|_2^2 + J_{obs}(z_k) \quad (40)$$

where $p_k = [x_k, y_k]^T$ denotes the robot position, $u_k = [\tau_r, \tau_l]^T$ is the control input vector, and $\Delta u_k = u_k - u_{k-1}$ represents the control rate.

The weighting matrices $W_{pos}, W_u, W_{\Delta u}, W_{\omega_r}, W_{\omega_l}$ are positive-definite diagonal matrices that tune the trade-off between tracking accuracy, energy consumption, actuator smoothness, and wheel speed regulation.

where $U = [u_0, u_1, \dots, u_{N_p-1}]^T$ is the sequence of right and left wheel torques $[\tau_r, \tau_l]^T$ to be optimized.

3.3 Quasi-Newton BFGS Optimization

The unconstrained control problem is solved using a custom implementation of the **Broyden-Fletcher-Goldfarb-Shanno (BFGS)** algorithm. At each iteration k , the search direction d_k

is computed by mapping the current gradient onto the approximate inverse Hessian manifold:

$$d_k = -H_k \nabla J(U_k) \quad (41)$$

where $H_k \in \mathbb{R}^{2N_p \times 2N_p}$ represents the inverse Hessian approximation, initialized as $H_0 = I$. To maintain stable descent and ensure the positive definiteness of the operator, H_k is updated via the **Sherman-Morrison** identity:

$$H_{k+1} = H_k + \left(1 + \frac{y_k^T H_k y_k}{s_k^T y_k}\right) \frac{s_k s_k^T}{s_k^T y_k} - \frac{H_k y_k s_k^T + s_k y_k^T H_k}{s_k^T y_k} \quad (42)$$

The displacement vectors are defined as the variation in control inputs $s_k = U_{k+1} - U_k$ and the corresponding change in cost gradients $y_k = \nabla J_{k+1} - \nabla J_k$.

3.4 Optimal Step Size via Strong Wolfe Conditions

To guarantee global convergence, the step size α_k is determined through a line search procedure designed to satisfy the **Strong Wolfe Conditions**. The optimization initiates with a unit step size ($\alpha_{init} = 1$), which is the ideal step for Newton-type methods. This value is iteratively refined using a **Bisection** method to find an α_k that ensures both sufficient cost decrease and curvature stability:

- **Sufficient Decrease (Armijo Condition):** $J(U_k + \alpha_k d_k) \leq J(U_k) + c_1 \alpha_k \nabla J_k^T d_k$
- **Curvature Condition:** $|\nabla J(U_k + \alpha_k d_k)^T d_k| \leq c_2 |\nabla J_k^T d_k|$

where the constants are selected as $c_1 = 10^{-4}$ and $c_2 = 0.9$ to provide a robust balance between convergence speed and numerical stability.

3.5 Numerical Gradient Computation

Numerical gradients are computed using the **Central Finite Differences** method to achieve a high-fidelity estimation of the cost surface. The partial derivatives with respect to each control input u_i are approximated as:

$$\frac{\partial J}{\partial u_i} \approx \frac{J(U + \tau e_i) - J(U - \tau e_i)}{2\tau} \quad (43)$$

where e_i is the i -th unit vector and the perturbation size is set to $\tau = 6 \times 10^{-6}$ to minimize the combined effect of truncation and floating-point round-off errors.

3.6 Implementation Workflow

The proposed control strategy is executed in a Receding Horizon Control (RHC) framework. The optimization problem is solved at each sampling instant to compute an open-loop optimal control sequence, while feedback is achieved by applying only the first element of this sequence. The systematic implementation is summarized in the following algorithm loop:

Algorithm 1: Receding Horizon Control with Quasi-Newton BFGS

1. Initialization:

- Set robot parameters *params* and prediction horizon N_p .

- Set initial state $z(0) \leftarrow z_0$ and initial control guess $U_{guess} \leftarrow \mathbf{0}_{2N_p \times 1}$.
- Define the obstacle map obs and target goals sequence $goals$.

2. Receding Horizon Loop (for $k = 0, 1, \dots, N - 1$):

- Goal Management:** If $\|z_k - goal_{current}\| < \text{tolerance}$, then $goal_{current} \leftarrow goal_{next}$.
- Optimal Control Computation:**
 - Solve $U^* \leftarrow \arg \min_U J_{N_p}(z_k, U)$ using the **Quasi-Newton BFGS** solver.
 - Use $U_{warm} \leftarrow [u_1^*, \dots, u_{N_p-1}^*, 0]^T$ as the starting point for the next iteration to enhance convergence.
- Control Application:**
 - Extract the first optimal input: $u_k \leftarrow u_0^*$.
 - Apply u_k to the accurate nonlinear model.
- Dynamics Integration:**
 - Update the state using the Runge-Kutta 4th Order (RK4) method: $z_{k+1} \leftarrow f_{RK4}(z_k, u_k, T_s)$.
- Shift Horizon:** $k \leftarrow k + 1$ and repeat from step (a).

3.7 Simulation Results

In this section, the performance of the proposed Receding Horizon Control (RHC) framework is evaluated using the accurate nonlinear dynamic model. The simulation environment consists of a 10×10 m workspace containing eight sequential goals and multiple static circular obstacles with a safety radius $R_{obs} = 0.6$ m.

Numerical and Implementation Parameters (Unconstrained Case)

The unconstrained nonlinear optimal control problem is solved at each sampling instant without imposing state or input constraints. The optimization is therefore driven solely by the objective function.

The main numerical parameters used in the simulations are summarized as follows:

- Sampling time: $T_s = 0.05$ s
- Prediction horizon: $N_p = 20$
- Total simulation time: $T = 16$ s
- Integration scheme: 4th-order Runge–Kutta (RK4)
- Maximum solver iterations: 250
- Optimality tolerance: 10^{-6}
- Step tolerance: 10^{-6}

3.7.1 Trajectory and Navigation Analysis

The robot's ability to navigate through the environment while maintaining safety is illustrated in Figure 5.

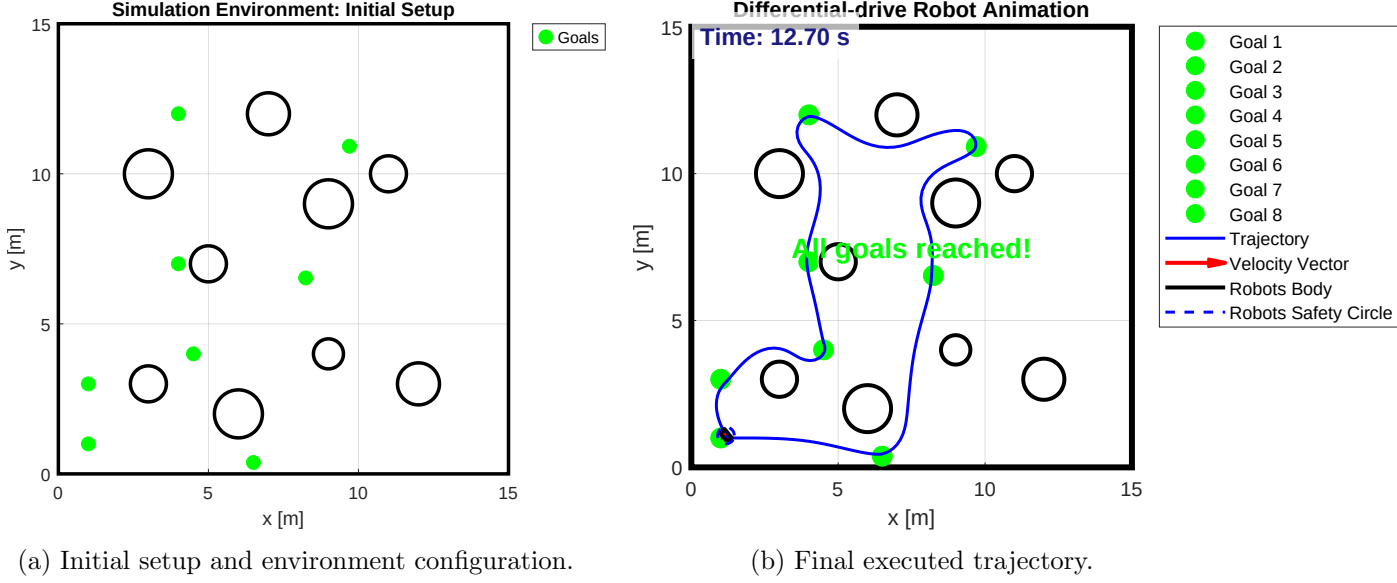
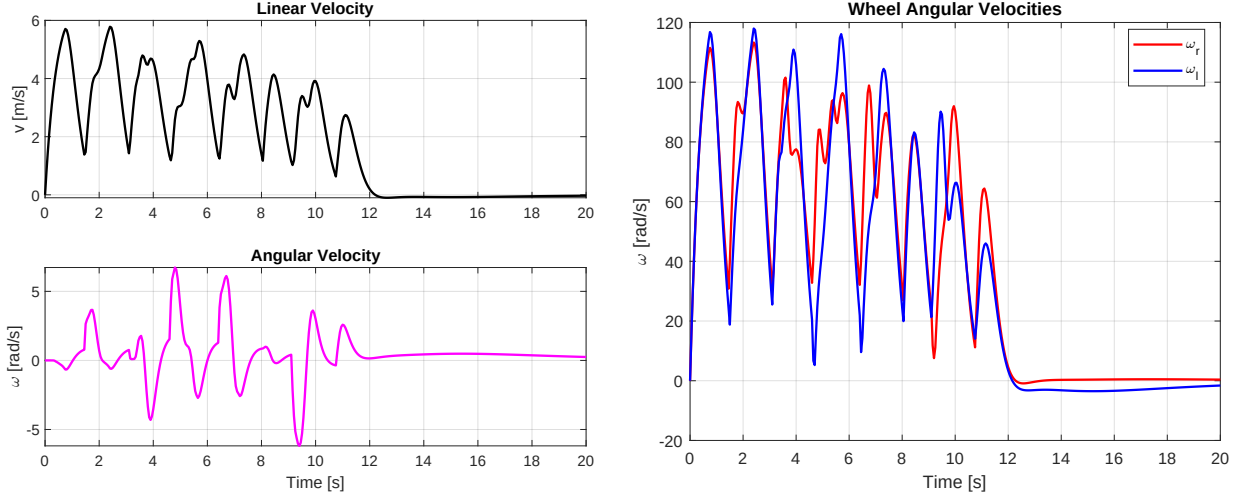


Figure 5: Robot navigation performance comparison. The initial environment setup (a) and the finalized trajectory (b) are presented in a consistent geometric perspective.

The results show that the robot successfully reaches all goals in the prescribed order. The trajectory remains smooth even in the neighborhood of obstacles, confirming that the barrier function parameters (α, β) provide a sufficient repulsive field without inducing oscillations.

3.7.2 Robot and Wheel Velocities Analysis

The relationship between platform-level velocities and wheel-level angular rates is presented in Figure 6.



(a) Robot linear (v) and angular (ω) velocities.

(b) Individual wheel angular velocities (ω_r, ω_l).

Figure 6: Kinematic and dynamic response comparison. The synchronized peaks in wheel velocities correspond to the linear acceleration phases, while the differences between ω_r and ω_l drive the steering maneuvers.

3.7.3 Actuator Dynamics and Constraints

The control effort required to achieve the trajectory is analyzed through the wheel torques. As shown in Figure 7, the torque signals τ_r and τ_l frequently saturate at the physical limit of ± 2.85 N m, particularly during the transition between goals where high angular acceleration is required to correct the heading angle θ .

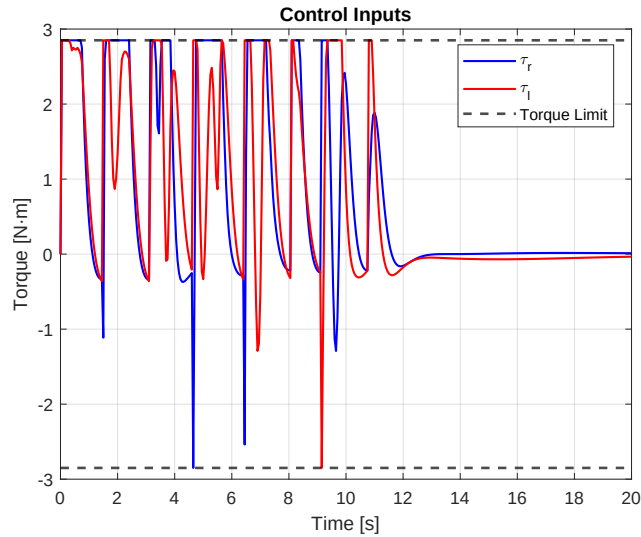


Figure 7: Control inputs (wheel torques) during the simulation. Dashed lines indicate actuator saturation limits ± 2.85 Nm per each.

3.7.4 Cost Function Convergence

The numerical stability of the BFGS optimizer is confirmed by the evolution of the cost in Figure 16. The total cost J converges rapidly at each sampling step. The behavior of minimization of the cost function can be seen as monotonically decreasing until the next goal is reached. It can

be stated that the BFGS algorithm finds the local minima at every step. The peaks in J_{total} occur exactly at the moment a new goal is assigned, as the tracking error e_{pos} momentarily increases before the solver computes a new optimal sequence to minimize the deviation.

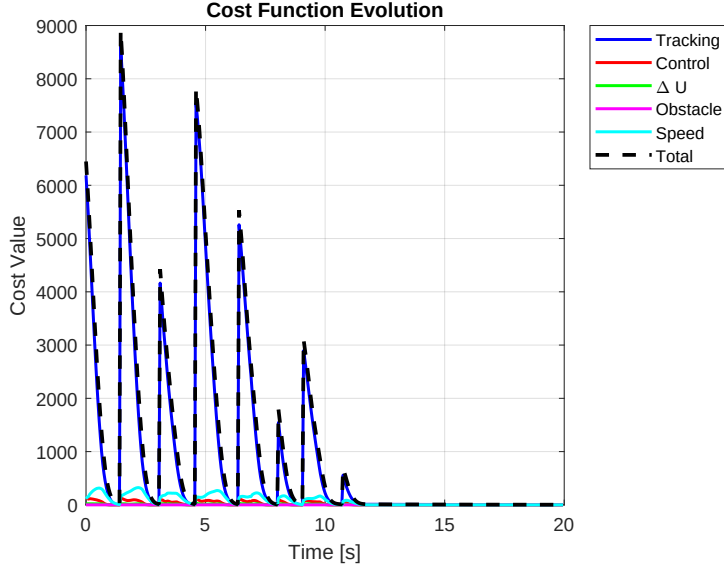


Figure 8: Evolution of the performance index components over time.

4 Part II: Constrained Optimal Control Problem

In the first part of this project, the trajectory tracking and obstacle avoidance task was formulated as an *unconstrained* optimal control problem by embedding the collision avoidance requirement into the cost function through an exponential barrier term of the form

$$J_{obs}(z) = \alpha \exp(-\beta d(z)), \quad (44)$$

where α and β are positive scalars and $d(z)$ denotes the minimum distance between the robot and the obstacle. Although this function is non-convex with respect to the state variables, the overall optimization landscape observed in simulations exhibited locally convex behavior around feasible trajectories, allowing the BFGS-based solver to converge reliably.

In this second phase, the problem is reformulated as an **explicitly constrained** Optimal Control Problem (OCP). Instead of relying on soft penalties, obstacle avoidance is now enforced through hard inequality constraints. This provides stronger safety guarantees and avoids the tuning sensitivity associated with barrier parameters α and β .

Static and Dynamic Obstacles While the unconstrained formulation considered only *circular static obstacles*, the constrained framework supports a richer set of environments:

- **Circular static obstacles**, identical to Part I but treated via explicit distance constraints;
- **Ellipsoidal and rectangular static obstacles**, where rectangles are approximated by smooth ellipsoids to preserve differentiability of the constraints;
- **Dynamic obstacles**, assumed to be circular, moving with *constant known velocity*. Their positions are assumed to be measurable at every sampling instant, which is consistent with realistic perception pipelines.

The new OCP is implemented using **CasADi**, which is selected due to:

- its symbolic representation enabling exact algorithmic differentiation of nonlinear dynamics and constraints,
- efficient generation of sparse NLP structures suitable for real-time MPC,
- seamless interfacing with IPOPT and other interior-point solvers.

4.1 Constrained MPC Formulation

At each sampling instant, a finite-horizon constrained optimization problem is solved in a receding horizon fashion. The exponential barrier term used in Part I,

$$\alpha \exp(-\beta d(z)),$$

is **removed** from the cost and replaced by explicit safety constraints.

Safe Distance Inequality and Linearization The natural collision avoidance condition for a circular obstacle with center O_i and radius R_i is

$$\|p_k - O_i\|_2 \geq R_i + r_{robot}. \quad (45)$$

This constraint is **non-convex** because the feasible set corresponds to the **exterior of a circle**. Direct use of this inequality would lead to a non-convex NLP with multiple local minima.

To preserve convexity within each MPC iteration, the constraint is linearized around the current state p_0 using a first-order Taylor expansion which will be shown later.

This linearization generates a **half-space approximation** of the original feasible region. The reasoning is that, locally, the tangent hyperplane at p_0 provides the closest convex inner approximation of the true safe set. Consequently:

- the resulting constraint is affine in the decision variables,
- the MPC problem becomes a smooth nonlinear program with convex safety constraints,
- real-time solution is significantly accelerated without sacrificing local safety.

Handling Dynamic Obstacles Dynamic obstacles are treated through a discrete-time Control Barrier Function (CBF). Under the assumptions:

- each obstacle is circular,
- it moves with constant known velocity,
- its position $O_j(t)$ is measurable at every time step,

$$O_j(t + kT_s) = O_j(t) + v_j kT_s,$$

which allows the MPC to anticipate future collisions and generate proactive avoidance maneuvers.

Overall, the constrained formulation replaces heuristic repulsive costs with **geometrically interpretable safety constraints**, providing improved robustness, easier tuning, and compatibility with modern NLP solvers through CasADi.

4.1.1 Objective Function and Constraints

The objective is to minimize the tracking error while ensuring control smoothness and actuator longevity. The performance index J is defined over the prediction horizon N_p :

$$\min_U \sum_{k=0}^{N_p-1} W_{pos} \|p_k - p_{goal}\|_2^2 + W_u \|u_k\|_2^2 + W_{\Delta u} \|\Delta u_k\|_2^2 + W_{\omega_r} \|\omega_{r,k}\|_2^2 + W_{\omega_l} \|\omega_{l,k}\|_2^2 \quad (46)$$

$$z_{k+1} = f_z(z_k, u_k) \quad (47a)$$

$$z_0 = z(t) \quad (47b)$$

$$h_{lin}(z_k, u_k) \geq 0 \quad (47c)$$

$$h_{cbf}(z_k, u_k) \geq 0 \quad (47d)$$

$$\underline{u} \leq u_k \leq \bar{u} \quad (47e)$$

where $p_k = [x_k, y_k]^T$ is the robot position, $u_k = [\tau_r, \tau_l]^T$ is the control input, and $\Delta u_k = u_k - u_{k-1}$ is the control rate.

The weight matrices $W_{pos}, W_u, W_{\Delta u}, W_{\omega_r}, W_{\omega_l}$ are diagonal positive-definite matrices tuned to balance aggressive tracking, energy efficiency, and actuator smoothness.

4.1.2 Basic Configurations on Circular Static Constraints

To mathematically define the collision avoidance strategy, we first establish the geometric relationship between the robot and a circular obstacle. Let the robot's state be represented by its center position $p_k \in \mathbb{R}^2$. The obstacle is modeled as a circle with center c_k and radius r_{obs} . To account for the physical dimensions of the robot, a safety radius r_b is introduced, effectively inflating the obstacle boundary.

The fundamental definitions for the static constraints are as follows:

- **Robot Position:** $p_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$ represents the center of the robot at time step k .
- **Obstacle Position:** $c_k = \begin{bmatrix} c_x \\ c_y \end{bmatrix}$ denotes the center of the k^{th} circular obstacle.
- **Vectorial Subtraction:** The relative position vector between the robot and the obstacle is defined as $\delta_k = p_k - c_k$.
- **Euclidean Distance:** The scalar distance between the centers is computed as $d_k = \|p_k - c_k\|_2$.
- **Unit Normal Vector:** The direction of the shortest path from the obstacle to the robot is given by:

$$n_{veck} = \frac{p_k - c_k}{\|p_k - c_k\|_2} = \frac{\delta_k}{d_k} \quad (48)$$

- **Nearest Point (Np_k):** The closest point on the inflated obstacle boundary relative to the robot's center is defined as:

$$Np_k = c_k + (r_{obs} + r_{robot})n_{veck} \quad (49)$$

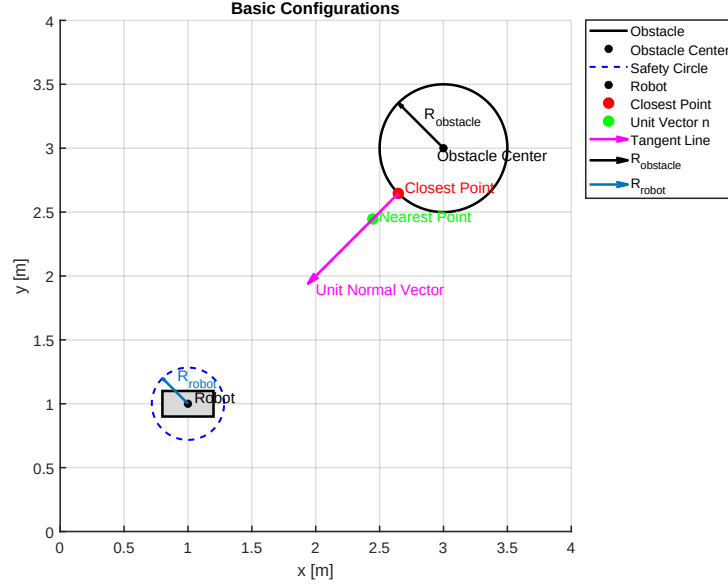


Figure 9: Geometric configuration of the robot and circular obstacle with associated vector definitions.

4.1.3 Linearized Static Obstacle Constraints

To maintain the convexity of the feasible search space for circular obstacles, a first-order Taylor expansion is applied. For an obstacle located at O_i with radius R_i , the non-convex distance constraint $\|p_k - O_i\|_2 \geq R_i + r_{robot}$ is linearized around the current position p_0 :

$$h_{lin}(p_k) \approx \|p_0 - O_i\|_2 + \frac{(p_0 - O_i)^T}{\|p_0 - O_i\|_2} (p_k - p_0) - (R_i + r_{robot}) \geq 0 \quad (50)$$

This approximation transforms the non-convex safe set into a half-plane constraint, significantly reducing the computational burden on the solver.

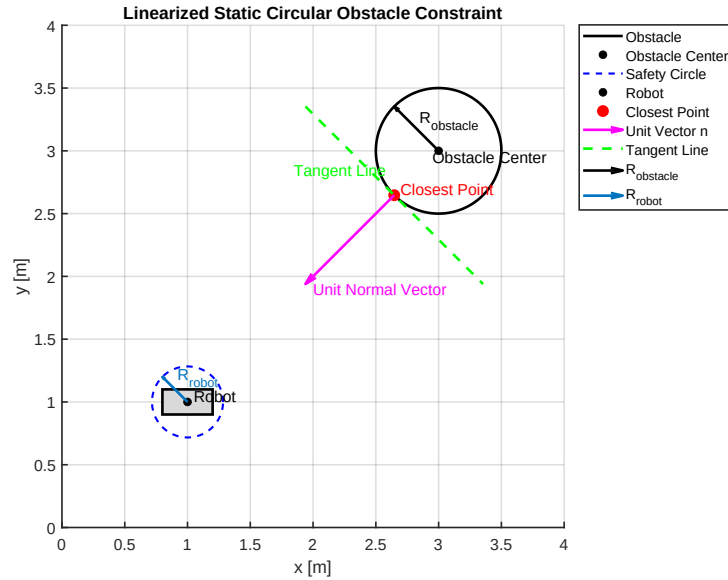


Figure 10: Linearization of the circular obstacle constraint using a tangent hyperplane at the current robot position.

4.1.4 Ellipsoid Approximation for Rectangular Obstacles

The use of smooth geometric approximations for non-differentiable obstacles has been widely adopted in the literature to preserve gradient continuity in optimization-based planners. In particular, the ellipsoidal representation of rectangular shapes has been employed in the context of Control Barrier Functions by Jian et al.[2], where the authors demonstrate that enclosing polytopic obstacles with differentiable ellipsoids enables the construction of reliable distance functions for predictive collision avoidance.

Motivated by this approach, we adopt an **ellipsoid approximation for rectangular static obstacles**. The goal is to replace the non-smooth rectangular geometry—whose corners lead to undefined gradients—with the smallest-area ellipse whose boundary passes through the four vertices of the rectangle. This guarantees:

- C^1 continuity of the obstacle constraint,
- availability of analytical gradients for CasADi,
- conservative but computationally tractable safety margins.

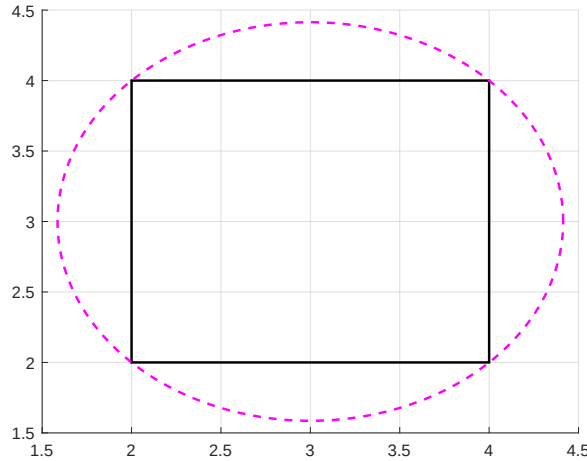


Figure 11: Minimum-area ellipse enclosing a rectangular obstacle with corner points lying on the ellipsoidal boundary.

4.1.5 Minimum-Area Ellipse Formulation

Consider a rectangle centered at (h, k) with width w and height h . An ellipse centered at the same point is defined as

$$\frac{(x - h)^2}{a^2} + \frac{(y - k)^2}{b^2} = 1. \quad (51)$$

To ensure that all four corners of the rectangle lie on the ellipse boundary, the semi-axes a and b are chosen as the solution of the following optimization problem:

$$\begin{aligned} \min_{a, b > 0} \quad & f(a, b) = \pi ab \\ \text{s.t.} \quad & \frac{(w/2)^2}{a^2} + \frac{(h/2)^2}{b^2} = 1. \end{aligned} \quad (52)$$

Solving this constrained minimization yields the closed-form optimal parameters

$$a^* = \frac{w\sqrt{2}}{2}, \quad b^* = \frac{h\sqrt{2}}{2}. \quad (53)$$

To account for the physical radius of the robot r_b , the **inflated safe ellipse** used in the MPC constraints becomes

$$a_{\text{safe}} = a^* + r_b, \quad b_{\text{safe}} = b^* + r_b. \quad (54)$$

This approximation allows rectangular obstacles to be treated within the same differentiable framework as circular ones, enabling a unified constraint representation in CasADi while maintaining conservative collision avoidance guarantees.

4.1.6 Linearization of the Ellipsoidal Constraint

Although the ellipsoidal approximation provides a smooth and differentiable representation of the rectangular obstacle, the constraint

$$h(p_k, c_k) = \left(\frac{x_k - c_x}{a_{\text{safe}}} \right)^2 + \left(\frac{y_k - c_y}{b_{\text{safe}}} \right)^2 - 1 \geq 0 \quad (55)$$

remains **non-convex** with respect to the robot position $p_k = [x_k, y_k]^T$. Direct inclusion of this constraint in the MPC would lead to a non-convex NLP and unreliable real-time convergence. For this reason, a first-order Taylor linearization around the current measured position p_0 is employed.

The gradient of the constraint with respect to the robot position is

$$\nabla h(p_k, c_k) = \begin{bmatrix} \frac{2(x_k - c_x)}{a_{\text{safe}}^2} \\ \frac{2(y_k - c_y)}{b_{\text{safe}}^2} \end{bmatrix}. \quad (56)$$

Using this gradient, the affine approximation at p_0 becomes

$$h(p_k, c_k) \approx h(p_0, c_k) + \nabla h(p_0, c_k)^T (p_k - p_0) \geq 0. \quad (57)$$

Expanding explicitly,

$$\begin{aligned} & \left(\frac{(x_0 - c_x)^2}{a_{\text{safe}}^2} + \frac{(y_0 - c_y)^2}{b_{\text{safe}}^2} - 1 \right) \\ & + \begin{bmatrix} \frac{2(x_0 - c_x)}{a_{\text{safe}}^2} \\ \frac{2(y_0 - c_y)}{b_{\text{safe}}^2} \end{bmatrix}^T (p_k - p_0) \geq 0. \end{aligned} \quad (58)$$

Rationale This linearization corresponds geometrically to constructing the **tangent hyperplane** to the ellipse at the closest point to the robot. The resulting half-space:

- is convex and affine in the decision variables,

- provides a conservative inner approximation of the true safe set,

As illustrated in Fig. 12, the normal vector $\nabla h(p_0, c_k)$ defines the direction of maximum increase of the constraint, which coincides with the unit distance vector from the ellipse toward the robot. By enforcing the linearized inequality at each MPC iteration, the robot is guaranteed to remain outside the inflated rectangular region while keeping the optimization problem tractable.

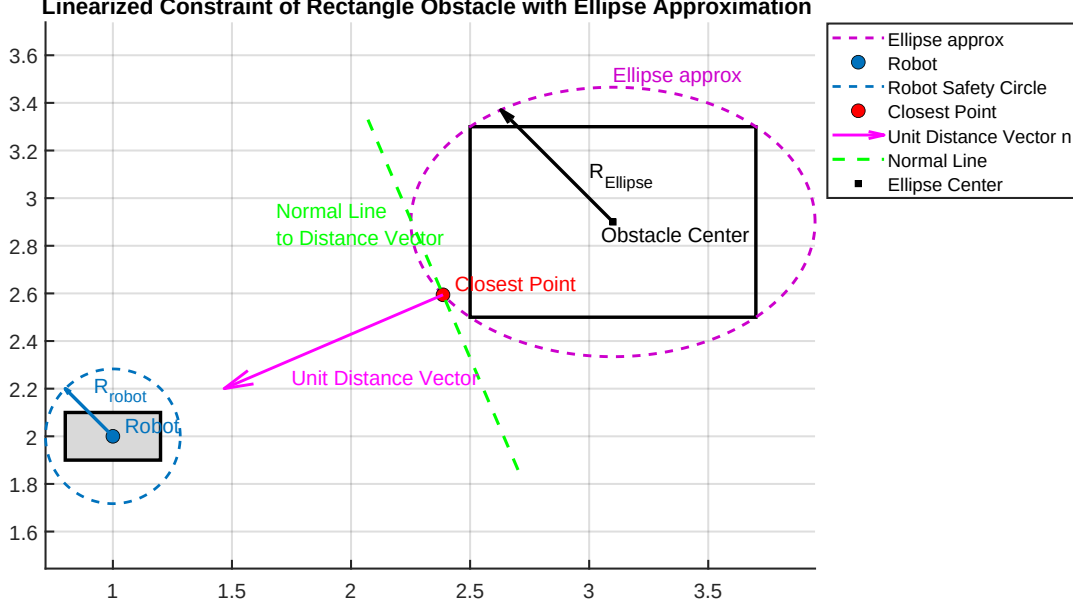


Figure 12

4.1.7 Dynamic Obstacle Avoidance via Control Barrier Functions (CBF)

For obstacles with time-varying states $O_j(t)$, a Control Barrier Function (CBF) framework is adopted to guarantee safety in a dynamically consistent manner. Unlike static obstacle linearization, CBFs enforce forward invariance of the safe set directly through system dynamics. In this formulation, we rely on two critical assumptions for dynamic entities:

A.1 Constant Velocity Assumption: Each dynamic obstacle moves with a **constant velocity vector** v_j over the prediction horizon.

A.2 Perfect State Measurement: The position $O_j(t)$ and velocity v_j of the obstacles are **perfectly measurable at each time step** t .

The safe set is defined as

$$\mathcal{C} = \{z \in \mathbb{R}^n \mid h(z, t) \geq 0\}, \quad (59)$$

where the safety function is

$$h(z, t) = \|p - O_j(t)\|_2 - d_{safe}, \quad (60)$$

with $p = [x, y]^T$ and $d_{safe} = R_j + r_{robot}$.

The objective is to guarantee forward invariance of $\mathcal{C}$, meaning that if $h(z(0), 0) \geq 0$, then $h(z(t), t) \geq 0$ for all future time.

4.1.8 Preliminaries: Lie Derivatives and Class- $\mathcal{K}$ Functions

Consider a nonlinear control-affine system

$$\dot{z} = f(z) + g(z)u. \quad (61)$$

Lie Derivative Definition:

Let $h : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuously differentiable.

The Lie derivative of h along vector field f is defined as

$$L_f h(z) = \frac{\partial h}{\partial z} f(z). \quad (62)$$

Similarly, the Lie derivative along g is

$$L_g h(z) = \frac{\partial h}{\partial z} g(z). \quad (63)$$

The time derivative of h along system trajectories is therefore

$$\dot{h}(z, u) = L_f h(z) + L_g h(z)u. \quad (64)$$

Relative Degree:

The safety function $h(z)$ has relative degree m if

$$L_g L_f^{i-1} h(z) = 0 \quad \text{for } i = 0, \dots, m-1,$$

and

$$L_g L_f^{m-1} h(z) \neq 0.$$

Class- $\mathcal{K}$ Function:

A continuous function $\alpha : [0, a) \rightarrow [0, \infty)$ is called a class- $\mathcal{K}$ function if:

- $\alpha(0) = 0$,
- $\alpha(s)$ is strictly increasing.

If additionally $\alpha(s) \rightarrow \infty$ as $s \rightarrow \infty$, it is called a class- $\mathcal{K}_\infty$ function.

4.1.9 Continuous-Time CBF Formulation

Consider the nonlinear control-affine system

$$\dot{z} = f(z) + g(z)u, \quad (65)$$

where

$$z = [x, y, \theta, \omega_r, \omega_l]^T, \quad u = [\tau_r, \tau_l]^T.$$

For relative degree one systems, the classical CBF condition [3] ensuring forward invariance is

$$\dot{h}(z, u) + \alpha(h(z)) \geq 0, \quad (66)$$

where $\alpha(\cdot)$ is a class- $\mathcal{K}$ function.

4.1.10 Relative Degree Analysis

For the differential-drive robot, the safety function depends only on the position:

$$h(z, t) = \|p - O_j(t)\|_2 - d_{safe}. \quad (67)$$

The first Lie derivative is

$$\dot{h} = \frac{(p - O_j)^T}{\|p - O_j\|} (\dot{p} - \dot{O}_j). \quad (68)$$

Since

$$\dot{p} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}, \quad v = \frac{r}{2}(\omega_r + \omega_l), \quad (69)$$

the control torques $u = [\tau_r, \tau_l]^T$ do not appear explicitly in $\dot{h}$.

Thus,

$$L_g h(z) = 0, \quad (70)$$

which implies that the safety function has relative degree greater than one.

Taking the second derivative,

$$\ddot{h} = L_f^2 h(z) + L_g L_f h(z) u, \quad (71)$$

where now the input appears through $\dot{\omega}_r$ and $\dot{\omega}_l$.

Therefore, the safety function has **relative degree two**.

4.1.11 High-Order (Second-Order) Continuous-Time CBF

For relative degree two systems, a High-Order Control Barrier Function (HOCBF) is required.

Define

$$\psi_0(z) = h(z), \quad (72)$$

$$\psi_1(z) = \dot{h}(z) + \alpha_1(h(z)). \quad (73)$$

Forward invariance is guaranteed if

$$\dot{\psi}_1(z, u) + \alpha_2(\psi_1(z)) \geq 0, \quad (74)$$

which yields

$$L_f^2 h(z) + L_g L_f h(z) u + \alpha_1 L_f h(z) + \alpha_2 \psi_1(z) \geq 0. \quad (75)$$

This inequality is affine in the control input u and can therefore be included as a constraint in the MPC optimization problem.

4.1.12 Discrete-Time CBF Formulation

Since MPC operates in discrete time with sampling time T_s , consider the discrete-time system

$$z_{k+1} = f(z_k, u_k). \quad (76)$$

Define

$$\Delta h(z_k) = h(z_{k+1}) - h(z_k). \quad (77)$$

The discrete-time CBF condition ensuring forward invariance is

$$\Delta h(z_k) + \alpha(h(z_k)) \geq 0, \quad (78)$$

or equivalently,

$$h(z_{k+1}) - h(z_k) + \alpha(h(z_k)) \geq 0. \quad (79)$$

4.1.13 Discrete-Time High-Order CBF (Relative Degree m)

If the safety function has relative degree $m > 1$, the input appears only after m forward differences.

The discrete-time HOCBF construction follows [4], define recursively:

$$\psi_0(z_k) := h(z_k), \quad (80)$$

$$\psi_1(z_k) := \Delta \psi_0(z_k) + \alpha_1(\psi_0(z_k)), \quad (81)$$

$$\psi_2(z_k) := \Delta \psi_1(z_k) + \alpha_2(\psi_1(z_k)), \quad (82)$$

$$\vdots \quad (83)$$

$$\psi_m(z_k) := \Delta \psi_{m-1}(z_k) + \alpha_m(\psi_{m-1}(z_k)), \quad (84)$$

where

$$\Delta \psi_i(z_k) = \psi_i(z_{k+1}) - \psi_i(z_k).$$

The discrete-time high-order CBF constraint is

$$\psi_m(z_k) \geq 0. \quad (85)$$

4.1.14 Relative Degree Two Case

For the differential-drive robot, $m = 2$. Thus,

$$\psi_0(z_k) = h(z_k), \quad (86)$$

$$\psi_1(z_k) = h(z_{k+1}) - h(z_k) + \alpha_1(h(z_k)), \quad (87)$$

$$\psi_2(z_k) = \psi_1(z_{k+1}) - \psi_1(z_k) + \alpha_2(\psi_1(z_k)). \quad (88)$$

The DT-HOCBF condition becomes

$$\psi_2(z_k) \geq 0. \quad (89)$$

4.1.15 Integration into the MPC Framework

The final constraint can be written compactly as

$$h_{CBF}(z_k, u_k) \geq 0, \quad (90)$$

and is enforced at each prediction step of the MPC together with static obstacle constraints:

$$\begin{aligned} h_{lin}(z_k, u_k) &\geq 0 \quad (\text{static obstacles}), \\ h_{CBF}(z_k, u_k) &\geq 0 \quad (\text{dynamic obstacles}). \end{aligned}$$

This guarantees:

- Forward invariance of the safety set,
- Compatibility with nonlinear MPC,
- Real-time feasibility due to affine dependence on control inputs,
- Safety-critical avoidance of moving obstacles.

4.2 Simulation Results

In this section, the performance of the proposed constrained NMPC framework is evaluated using the accurate nonlinear dynamic model together with explicit static and dynamic safety constraints. Unlike Part I, obstacle avoidance is now enforced through hard linearized constraints and discrete-time High-Order Control Barrier Functions (DT-HOCBFs), rather than cost-based penalties.

The simulation environment is identical to the unconstrained case in terms of goal configuration, but now includes both circular and rectangular static obstacles as well as moving circular obstacles handled via CBF constraints.

4.2.1 Trajectory and Constraint Satisfaction

The resulting constrained trajectory is illustrated in Figure 13.

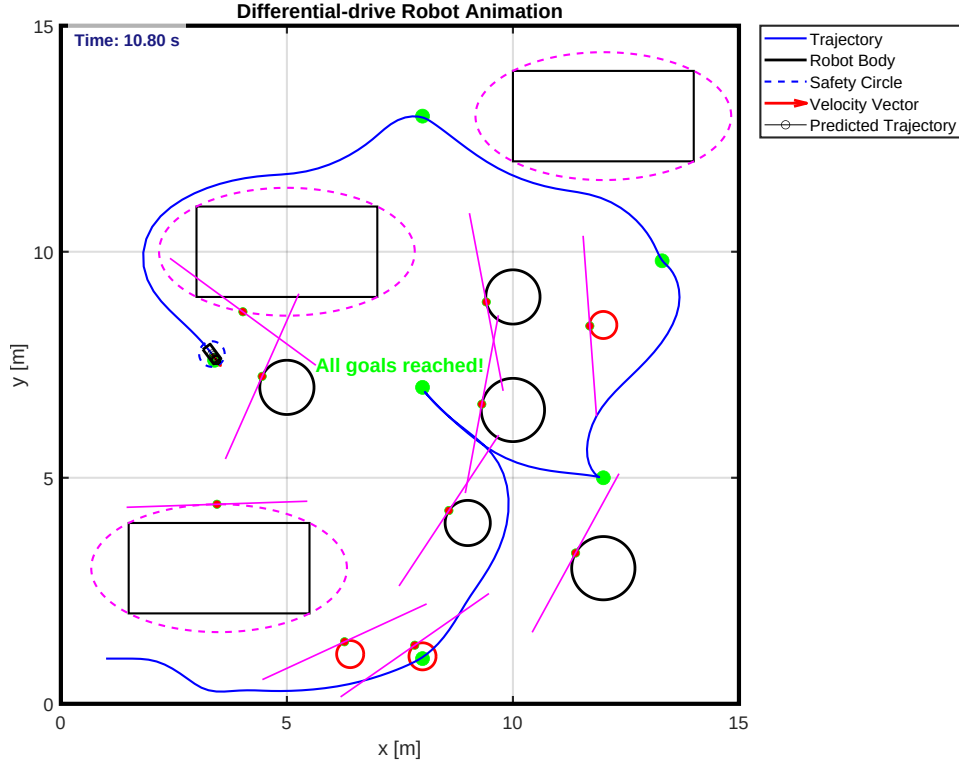


Figure 13: Final executed trajectory under constrained NMPC with static and dynamic obstacle avoidance.

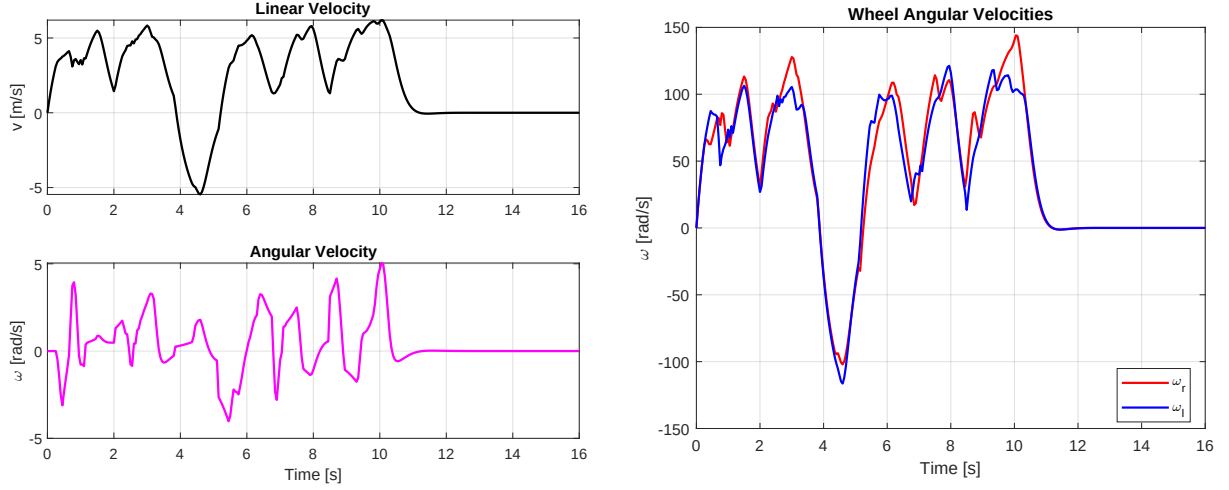
The results show that the robot successfully reaches all prescribed goals while strictly respecting the safety margins.

Compared to the unconstrained barrier-based approach, the trajectory exhibits slightly more conservative maneuvers near obstacles. This is expected, as hard constraints prevent the optimizer from temporarily entering regions of low safety margin, which was sometimes tolerated in the soft-penalty formulation.

Importantly, no constraint violations are observed throughout the simulation, confirming forward invariance of the safe set under the DT-HOCBF formulation.

4.2.2 Robot and Wheel Velocities Analysis

The platform-level linear and angular velocities together with the corresponding wheel angular velocities are shown in Figure 14.



(a) Robot linear (v) and angular (ω) velocities.

(b) Individual wheel angular velocities (ω_r, ω_l).

Figure 14: Platform and wheel velocity profiles under constrained NMPC with static and dynamic obstacle avoidance.

The velocity profiles remain smooth despite the presence of hard inequality constraints. When the robot approaches static or dynamic obstacles, a clear modulation in angular velocity ω is observed, capturing steering corrections generated by the linearized obstacle constraints and DT-HOCBF safety conditions.

Compared to the unconstrained barrier-based formulation in Part I, the constrained NMPC produces more structured and predictive avoidance behavior, avoiding sudden changes exception of between 4-6 seconds.

Confirming that the quadratic penalties on control effort and rate successfully regularize the solution even in the presence of strict safety constraints.

4.2.3 Actuator Torques and Feasibility of Hard Constraint

The actuator torques are depicted in Figure 15.

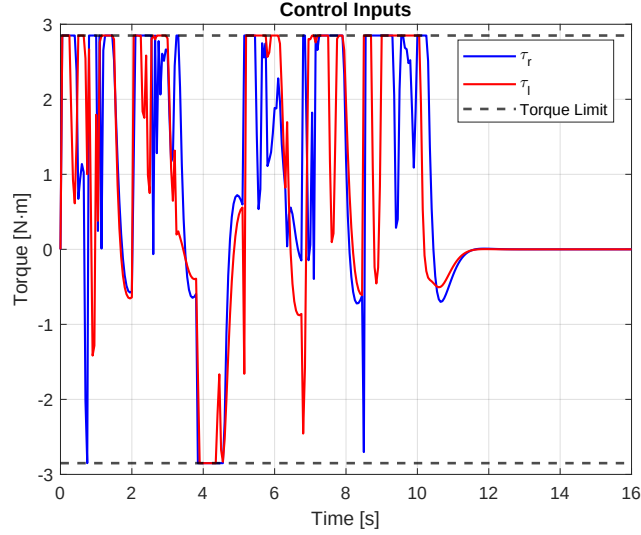


Figure 15: Wheel torques under constrained NMPC. Dashed lines indicate actuator saturation limits ± 2.85 Nm per each.

Compared to the unconstrained case, torque signals are generally less oscillatory near obstacles. However, during rapid heading corrections—especially in dynamic obstacle scenarios—brief torque saturation occurs. This behavior is consistent with the high-order CBF constraint, which enforces second-order safety and may require sharper corrective accelerations.

Despite these corrections, the control inputs remain within physical bounds at all times, validating the feasibility of the constrained formulation.

4.2.4 Cost Function Convergence

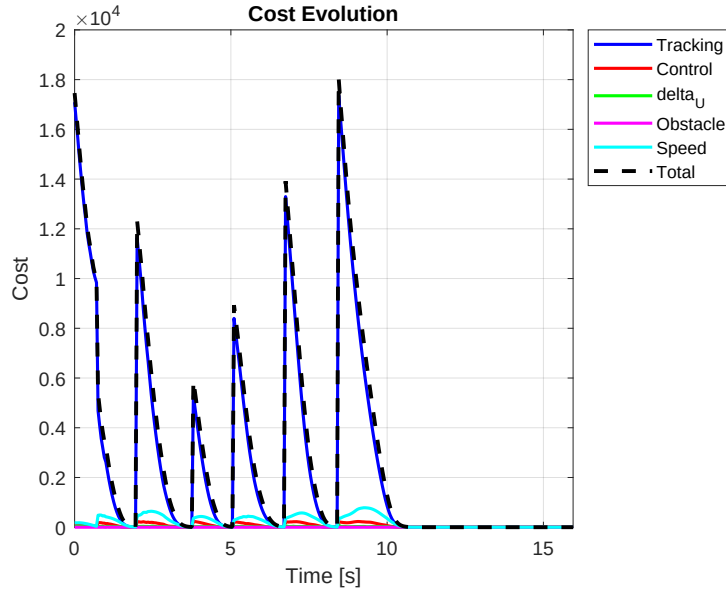


Figure 16: Evolution of the performance index components over time.

Similar to the unconstrained case, the constrained formulation also exhibits local convexity properties around the solution. As observed in the cost function evolution plot (see Fig. 16), the objective value decreases monotonically throughout the prediction horizon until the goal

configuration is reached. This monotonic decrease indicates that the optimization problem remains well-conditioned despite the presence of linearized obstacle constraints and DT-HOCBF safety conditions. Although global optimality cannot be guaranteed due to the nonlinear and constrained structure of the problem, the observed behavior confirms convergence to a local minimum (or its neighborhood) that satisfies both performance and safety requirements.

4.2.5 Comparison with Unconstrained Formulation

A qualitative comparison between Part I and Part II reveals several important observations:

- The constrained NMPC provides **strict safety guarantees**, whereas the barrier-based formulation offers only soft safety enforcement.
- Trajectories in Part II are slightly more conservative but geometrically cleaner near obstacle boundaries.
- Dynamic obstacle avoidance using DT-HOCBF generates anticipatory behavior rather than reactive repulsion.
- Numerical stability remains satisfactory despite the inclusion of nonlinear safety constraints.

Overall, the constrained formulation demonstrates improved robustness and safety interpretability while maintaining acceptable computational performance for real-time implementation.

4.2.6 Artificial LiDAR Integration

To enhance environmental awareness, an artificial LiDAR range is implemented. The robot only considers obstacles within a sensing radius D_{sensor} . This mimics real-world sensor limitations and reduces the number of active constraints in the optimization problem:

$$\mathcal{O}_{active} = \{O_i \mid \|p_0 - O_i\|_2 \leq D_{sensor}\} \quad (91)$$

5 Numerical Optimization Strategy

The resulting Nonlinear Program (NLP) is solved in a receding horizon fashion within a Nonlinear Model Predictive Control (NMPC) framework. Both **Single Shooting** and **Multiple Shooting** strategies were evaluated during the development phase.

In the Single Shooting approach, the optimization variables consist solely of the control input sequence over the prediction horizon. The system dynamics are integrated forward in time (here using a fourth-order Runge–Kutta scheme) to obtain the predicted state trajectory. This results in a lower-dimensional optimization problem compared to Multiple Shooting, since intermediate states are not treated as decision variables.

Multiple Shooting, on the other hand, augments the decision vector with intermediate state variables and enforces dynamic consistency through equality constraints. While this typically improves numerical robustness for long horizons and highly nonlinear dynamics—by preventing the accumulation of integration errors—it significantly increases the problem size and computational burden.

All numerical results reported in this paper were obtained using the Single Shooting formulation. For the considered horizon length and sampling time, Single Shooting provided a favorable trade-off between computational efficiency and numerical stability. In particular, it reduced the dimensionality of the NLP, simplified constraint handling, and enabled efficient warm-starting between consecutive MPC iterations.

Solver and Implementation Details

The constrained NLP (Part II) is solved using `fmincon()` in MATLAB with the Sequential Quadratic Programming (SQP) algorithm. The SQP method is particularly suitable for constrained nonlinear problems, as it solves a sequence of quadratic programming subproblems that locally approximate the NLP.

The main implementation parameters are:

- Sampling time: $T_s = 0.05$ s
- Prediction horizon: $N_p = 20$
- Simulation duration: $T = 16$ s
- Integration method: 4th-order Runge–Kutta
- Maximum SQP iterations: 250
- Optimality tolerance: 10^{-6}
- Step tolerance: 10^{-6}
- CBF coefficients: $\alpha_1 = 0.15$ and $\alpha_2 = 0.10$

At each MPC step, the previously optimal control sequence is shifted and used as an initial guess (warm start), significantly accelerating convergence of the SQP iterations. Nonlinear inequality constraints include obstacle avoidance constraints and DT-HOCBF safety conditions, while box constraints enforce actuator saturation limits.

This implementation ensures real-time feasibility for the selected horizon length while maintaining constraint satisfaction and stable closed-loop performance.

6 Conclusions

The second phase of the project highlighted the advantages of a transition to a constrained Model Predictive Control (MPC) environment. By linearizing static obstacle constraints and integrating Control Barrier Functions (CBFs), the system achieved a higher degree of safety and reliability, particularly when navigating toward multiple goals in cluttered workspaces. Our comparative analysis between simplified and accurate nonlinear dynamic models revealed that while simplified models are computationally attractive, accurate models integrated with high-order methods like Runge-Kutta 4 are essential for maintaining precision at higher sampling times. The results confirmed that the Receding Horizon Control (RHC) strategy, supported by a warm-started BFGS quasi-Newton solver, converges efficiently to optimal control inputs even under torque saturation limits.

Despite the success of the current implementation, several avenues for future research remain. The current framework could be further enhanced by the following:

- **Guaranteed Stability:** Future iterations should incorporate suitable terminal costs and terminal sets within the NMPC formulation to provide formal proofs of recursive feasibility and asymptotic stability.
- **Advanced Solvers:** To decrease computational latency, the use of specialized optimization toolboxes such as CasADi or CVX could be explored for real-time applications.
- **Complex Navigation:** The integration of global path planning algorithms like Rapidly-exploring Random Trees (RRT) or Artificial Potential Fields (APF) could be used to initialize the local MPC, preventing the robot from becoming trapped in local minima in non-convex environments.
- **Multi-Robot Systems:** The proposed Control Barrier Function approach can be extended to multi-robot scenarios, utilizing Optimal Reciprocal Collision Avoidance (ORCA) to ensure safety in collaborative or competitive tasks.

In conclusion, the project successfully bridge the gap between theoretical numerical optimization and practical robotic control. The developed framework provides a robustness for autonomous systems required to operate safely and optimally in realistic, dynamic environments.

References

- [1] R. Dhaouadi and A. A. Hatab, “Dynamic modelling of differential-drive mobile robots,” *International Journal of Advanced Robotic Systems*, vol. 10, p. Article 263, 2013.
- [2] Z. Jian, Z. Yan, X. Lei, Z. Lu, B. Lan, X. Wang, and B. Liang, “Dynamic control barrier function-based model predictive control for safety-critical obstacle-avoidance of mobile robots,” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3679–3685, 2023.
- [3] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *Proceedings of the 18th European Control Conference (ECC)*, pp. 3420–3431, 2019.
- [4] Y. Xiong, D.-H. Zhai, M. Tavakoli, and Y. Xia, “Discrete-time control barrier function: High-order case and adaptive case,” *IEEE Transactions on Cybernetics*, vol. 53, no. 5, pp. 3231–3239, 2023.